

Estimation Engine

Last updated: 2026-05-29

The estimation engine lets an organisation define a matrix of expected work durations keyed on org-specific production variables. When the scheduler generates work for an asset, it looks up the asset's field values against this matrix to produce day-count estimates for each workflow step.

Org-scoped. The stack was originally studio-only. As of migration `20260529000001` it is owned by **either a studio or a vendor** (dual nullable FK `studio_id` / `vendor_id`, exactly one set, with a generated `owner_key`). Studio behaviour is byte-for-byte unchanged. Vendors author their matrix through the same OrgHub UI and can additionally keep per-studio-link rate overrides and share frozen snapshots with studios, see [Vendor Estimate Sharing](#).

Concepts

Workflow step: a named production task (e.g., "Concept Art", "Rigging", "QA") with an optional craft label and zero or more dependency steps. Defined per studio in `workflow_steps`. Steps have a topological ordering enforced by `workflow_step_dependencies`.

Variable fields: the source-tool fields whose values drive the estimate matrix. A studio chooses which fields are variables (e.g., "Asset Type", "Complexity"). The combination of all variable field values for a given asset determines which matrix row applies.

Estimate matrix: a table of `(workflow_step × variable_values) → estimate_days`. Each cell holds the number of working days expected for that step on an asset with those variable values. Cells can be null (unknown) or 0 (this step doesn't apply).

Combination: a unique ordered set of variable field values (e.g., `{Asset Type: "Character", Complexity: "High"}`). Each distinct combination becomes a column group in the matrix UI.

Data Model

All three tables carry the dual-owner columns (`studio_id` nullable, `vendor_id` nullable, `check (num_nonnulls(studio_id, vendor_id) = 1)`, generated `owner_key uuid = coalesce(studio_id, vendor_id)`). `owner_key` is the single uniqueness arbiter and the PostgREST `on_conflict` target. Below, "owner" means whichever of studio/vendor owns the row.

workflow_steps

id	uuid PK	
studio_id	uuid → studios	(nullable)
vendor_id	uuid → vendors	(nullable)
owner_key	uuid	(generated: coalesce(studio_id, vendor_id))
name	text NOT NULL	
craft	text	(optional grouping label, e.g., "Art", "Tech")
created_at, updated_at		

Unique index: (owner_key, airtable_template_id).

workflow_step_dependencies

step_id	uuid → workflow_steps (PK)
depends_on_step_id	uuid → workflow_steps (PK)

An edge (step_id, depends_on_step_id) means *step_id* needs *depends_on_step_id* first. ON DELETE CASCADE. Cycles are rejected server-side, see [Dependency cycle prevention](#).

estimate_config

id	uuid PK	(surrogate; was studio_id before org-scoping)
studio_id	uuid → studios	(nullable)
vendor_id	uuid → vendors	(nullable)
owner_key	uuid	(generated; unique)
variable_fields	text[]	(ordered list of source field names)
updated_at		

One row per owner. variable_fields determines which source asset fields are used as matrix axes.

estimate_matrix

id	uuid PK	
studio_id	uuid → studios	(nullable)
vendor_id	uuid → vendors	(nullable)
owner_key	uuid	(generated)
workflow_step_id	uuid → workflow_steps	
variable_values	jsonb	({field_name: value, ...})
link_id	uuid → studio_vendor_links	(nullable; vendor-only; NULL = base, set = per)
estimate_days	numeric	

Unique index: (owner_key, workflow_step_id, variable_values, link_id) **NULLS NOT DISTINCT** (so base rows, link_id IS NULL, collide). check (link_id is null or vendor_id is not null). GIN index on variable_values for fast lookup. The link_id override layer and the base⊕override **effective matrix** are covered in [Vendor Estimate Sharing](#).

Setup Flow

1. Define workflow steps

Owners build their step library on the Workflows page (`frontend/src/pages/Workflows.jsx`): - Add/edit/delete steps with name, craft, and dependency checkboxes - Steps are displayed grouped by craft, with "Needs" dependency pills, ordered by a client-side topological sort (`topoSort()`) - The edit modal disables any dependency that would close a cycle; the API rejects it regardless , see [Dependency cycle prevention](#) - CSV export available for bulk review

2. Create the matrix

`POST /api/setup/create-matrix-pg` initialises the matrix for a studio: 1. Takes `variable_fields` (list of source field names the studio has chosen) 2. Scans synced assets to enumerate all unique value combinations for those fields 3. Creates an `estimate_config` row 4. Creates one `estimate_matrix` row per `(step × combination)` with `estimate_days = null`

A "Default" combination is always included for assets whose field values don't match any explicit combination.

3. Fill in estimates

The Estimates page (`frontend/src/pages/Estimates.jsx`) renders the matrix as a spreadsheet: - **Rows:** workflow steps, sorted topologically, with step name, craft, and dependency labels in sticky left columns - **Columns:** one column group per variable combination; a header row shows the field name, a second header row shows the value - **Cells:** numeric inputs (min 0, step 0.5 days); editable inline

Cell saves happen on blur or Enter: `PATCH /api/setup/matrix-cell` upserts the single `(step, variable_values)` row. Keyboard shortcut: Escape reverts a cell to its saved value without saving.

Estimate Resolution

When the scheduler generates work for an asset, it looks up the asset's variable field values from `replicated_assets.meta` , builds a `variable_values` dict, and queries `estimate_matrix` for a matching row per workflow step.

Lookup priority: 1. Exact match on `variable_values` 2. `Default` combination (always present as a fallback) 3. `null` , no estimate (step is generated but with no day count)

The `null` vs `0` distinction is meaningful: `null` means the estimate is unknown or not yet entered; `0` means the step genuinely has zero duration for this combination (e.g., a simple prop that skips rigging).

Dependency cycle prevention

A circular dependency (A needs B needs ... needs A) is impossible to schedule, so it is rejected at two layers. Cycles can only be introduced on **update**, a brand-new step has no incoming edges.

- **API (authoritative).** `routes/workflow_steps.py` `_validate_deps()` runs on create and update. It rejects, with a `400`, dependencies that aren't the caller's own steps (cross-tenant), a self-dependency, and any change that would form a cycle (the error names the offending chain). On update it runs **before** the destructive delete-and-reinsert of dependency rows, so invalid input leaves the data untouched. Cycle detection is a pure, unit-tested helper, `lib/workflow_graph.py` `detect_cycle()` (`scripts/test_workflow_graph.py`).
- **UI (convenience).** The Workflows edit modal computes the transitive set of steps that depend on the step being edited and disables those checkboxes (selecting one would close a loop).

API Reference

Workflow-step endpoints are under `/api/workflow-steps`; matrix endpoints under `/api/setup`. Resolved to the calling user's owner (studio or vendor) via `resolve_owner()`.

Method	Path	Description
GET	<code>/api/workflow-steps</code>	List workflow steps with <code>depends_on</code> / <code>depended_by</code>
POST	<code>/api/workflow-steps</code>	Create a step (validates dependencies)
PATCH	<code>/api/workflow-steps/{id}</code>	Update name, craft, or dependencies (rejects cycles)
DELETE	<code>/api/workflow-steps/{id}</code>	Delete a step
DELETE	<code>/api/workflow-steps/bulk</code>	Delete multiple steps
GET	<code>/api/workflow-steps/csv</code>	Export steps + dependencies + crafts as CSV
GET	<code>/api/setup/matrix-table-pg</code>	Fetch formatted matrix; <code>?linkId=</code> returns the vendor effective matrix
POST	<code>/api/setup/create-matrix-pg</code>	Initialize matrix from variable fields + asset combinations
PATCH	<code>/api/setup/matrix-cell</code>	Upsert a single estimate cell (optional <code>link_id</code> for a vendor override)
POST	<code>/api/setup/randomize-matrix</code>	Admin: bulk-fill null/zero cells with random estimates

For the vendor estimate-share endpoints (`/api/estimate-shares/*`) see [Vendor Estimate Sharing](#).

Known Gaps

Variable fields are not linked to source_field_mappings: `estimate_config.variable_fields` stores raw field names. If an owner renames a field in their source tool and re-syncs, the variable field names in the matrix may become stale without detection.

No dead-end / unreachable-chain detection: cycle prevention is enforced (above), but a step whose prerequisites can never all complete (a distinct kind of invalid configuration) is not yet detected.